ASSIGNMENT 15.5

2303A51330
Kashireddy Akshitha
Batch 20

Task 1 – Student Attendance API
Task:
Create a RESTful API for student attendance tracking.
Instructions:
• Endpoints required:
o GET /attendance → View attendance records
o POST /attendance → Mark attendance
o PUT /attendance/{id} → Update attendance record
o DELETE /attendance/{id} → Remove attendance entry
• Ensure JSON-based request and response handling.
Expected Output:
• API for maintaining and updating attendance records.

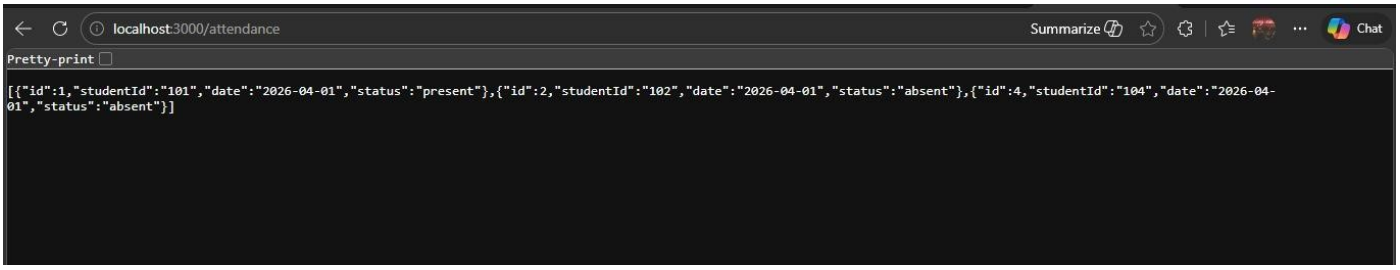
CODE:

```
const express = require('express');
const app = express();

// Middleware to parse incoming JSON requests
app.use(express.json());

// In-memory storage for attendance records
let attendanceRecords = [];
let nextId = 1;

// Root route to confirm the API is running
app.get('/', (req, res) => {
 res.status(200).send('Attendance API is running');
});

// GET /attendance - return all attendance records
app.get('/attendance', (req, res) => {
 res.status(200).json(attendanceRecords);
});

// POST /attendance - add a new attendance record
app.post('/attendance', (req, res) => {
 const { studentId, date, status } = req.body;

 if (!studentId || !date || !status) {
```

```
  return res.status(400).json({
    message: 'Missing required fields: studentId, date, and status are all required',
  });
 }

 const normalizedStatus = status.toString().trim().toLowerCase();
 if (normalizedStatus !== 'present' ss normalizedStatus !== 'absent') {
  return res.status(400).json({
    message: "Status must be either 'present' or 'absent'",
  });
 }

 const newRecord = {
  id: nextId++,
  studentId,
  date,
  status: normalizedStatus,
 };

 attendanceRecords.push(newRecord);
 res.status(201).json(newRecord);
});

// PUT /attendance/:id - update an existing attendance record
app.put('/attendance/:id', (req, res) => {
 const recordId = Number(req.params.id);
 const { studentId, date, status } = req.body;

 const record = attendanceRecords.find((item) => item.id === recordId);
 if (!record) {
  return res.status(404).json({ message: 'Attendance record not found' });
 }

 if (!studentId || !date || !status) {
  return res.status(400).json({
    message: 'Missing required fields: studentId, date, and status are all required',
  });
 }

 const normalizedStatus = status.toString().trim().toLowerCase();
 if (normalizedStatus !== 'present' ss normalizedStatus !== 'absent') {
  return res.status(400).json({
    message: "Status must be either 'present' or 'absent'",
  });
 }
```

```
 record.studentId = studentId;
 record.date = date;
 record.status = normalizedStatus;

 res.status(200).json(record);
});

// DELETE /attendance/:id - remove an attendance record
app.delete('/attendance/:id', (req, res) => {
 const recordId = Number(req.params.id);
 const index = attendanceRecords.findIndex((item) => item.id === recordId);

 if (index === -1) {
  return res.status(404).json({ message: 'Attendance record not found' });
 }

 const deletedRecord = attendanceRecords.splice(index, 1)[0];
 res.status(200).json(deletedRecord);
});

// Start Express server
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
 console.log(`Attendance API server is running on port ${PORT}`);
});
```

OUTPUT:



Task 2 – Inventory Management API

Task:

Develop a REST API for warehouse inventory management.

Instructions:

- Endpoints required:
  - GET /inventory → List all items
  - POST /inventory → Add new item
  - PUT /inventory/{id} → Update stock quantity
  - DELETE /inventory/{id} → Remove an item
- Implement validation for stock values.

Expected Output:
• API capable of managing inventory stock levels.

PROMPT:
//Create a RESTful API for warehouse inventory management using Node.js and Express. The API should handle JSON requests and responses and store data in an in-memory array. Each inventory item must include id, itemName, quantity, and price. Implement the following endpoints: GET /inventory to return all items, POST /inventory to add a new item with validation (itemName required, quantity must be a non-negative number, price must be positive), PUT /inventory/:id to update the stock quantity of an existing item with proper validation and error handling if the item is not found, and DELETE /inventory/:id to remove an item. Include a root route GET / that returns a simple message like 'Inventory API is running'. Use express.json() middleware, proper HTTP status codes (200, 201, 400, 404), and add clear comments so the code is easy to understand and ready to run in a single server.js file

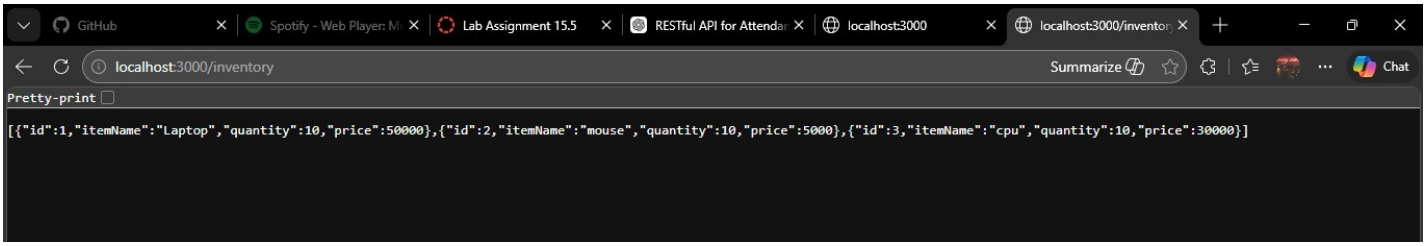
CODE:

```
console.log("Inventory Server starting...");

const express = require('express');
const app = express();

app.use(express.json());

let inventory = [];

// Home route
app.get('/', (req, res) => {
  res.send("Inventory API is running...");
});

// GET /inventory → List all items
app.get('/inventory', (req, res) => {
  res.json(inventory);
});

// POST /inventory → Add new item
app.post('/inventory', (req, res) => {
  const { itemName, quantity, price } = req.body;

  // Validation
  if (!itemName || quantity === undefined || price === undefined) {
    return res.status(400).json({ message: "All fields are required" });
  }
```

```
  if (quantity < 0 || price <= 0) {
    return res.status(400).json({ message: "Invalid stock or price value" });
  }

  const newItem = {
    id: inventory.length + 1,
    itemName,
    quantity,
    price
  };

  inventory.push(newItem);
  res.status(201).json(newItem);
});

// PUT /inventory/:id → Update stock quantity
app.put('/inventory/:id', (req, res) => {
  const itemId = parseInt(req.params.id);
  const { quantity } = req.body;

  if (quantity === undefined || quantity < 0) {
    return res.status(400).json({ message: "Invalid quantity value" });
  }

  const item = inventory.find(i => i.id === itemId);

  if (!item) {
    return res.status(404).json({ message: "Item not found" });
  }

  item.quantity = quantity;

  res.json(item);
});

// DELETE /inventory/:id → Remove item
app.delete('/inventory/:id', (req, res) => {
  const itemId = parseInt(req.params.id);

  const index = inventory.findIndex(i => i.id === itemId);

  if (index === -1) {
    return res.status(404).json({ message: "Item not found" });
  }
```

```
  const deletedItem = inventory.splice(index, 1);

  res.json(deletedItem[0]);
});

// Start server
const PORT = 3000;

app.listen(PORT, () => {
  console.log("Server is running on port " + PORT);
});
```

OUTPUT:



Task 3 – Hotel Room Booking API

Task:

Develop a RESTful API for hotel room booking.

Instructions:

• Endpoints required:

o GET /rooms → View available rooms

o POST /rooms/book → Book a room

o GET /rooms/{id} → View booking details

o DELETE /rooms/{id} → Cancel booking

• Implement error handling for unavailable rooms.

Expected Output:

• API simulating hotel room booking operations.

PROMPT:

//Create a RESTful API for a hotel room booking system using Node.js and Express. The API should handle JSON requests and responses and store data in an in-memory array. Each room should have fields like id, roomNumber, type, price, and isBooked (boolean). Implement the following endpoints: GET /rooms to list all available rooms (only those not booked), POST /rooms/book to book a room by providing room id and customer details, GET /rooms/:id to view booking details of a specific room, and DELETE /rooms/:id to cancel a booking and mark the room as available again. Include proper error handling such as returning a message if a room is already booked or not found. Use express.json() middleware, appropriate HTTP status codes (200, 201, 400, 404), and include a root route GET / that returns 'Hotel Booking API is running'. Ensure the code is clean, well-commented, and ready to run in a single server.js file.

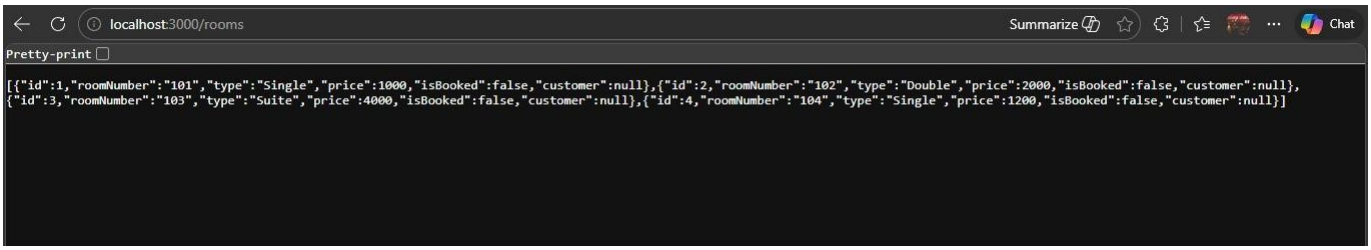
CODE:

```
console.log("Hotel Booking Server starting...");

const express = require('express');
const app = express();

app.use(express.json());

// Sample rooms data
let rooms = [
  { id: 1, roomNumber: "101", type: "Single", price: 1000, isBooked: false, customer: null },
  { id: 2, roomNumber: "102", type: "Double", price: 2000, isBooked: false, customer: null },
  { id: 3, roomNumber: "103", type: "Suite", price: 4000, isBooked: false, customer: null },
  { id: 4, roomNumber: "104", type: "Single", price: 1200, isBooked: false, customer: null }
];

// Home route
app.get('/', (req, res) => {
  res.send("Hotel Booking API is running...");
});

// GET /rooms → View available rooms
app.get('/rooms', (req, res) => {
  const availableRooms = rooms.filter(room => !room.isBooked);
  res.json(availableRooms);
});

// POST /rooms/book → Book a room
app.post('/rooms/book', (req, res) => {
  const { roomId, customerName } = req.body;

  if (!roomId || !customerName) {
    return res.status(400).json({ message: "roomId and customerName are required" });
  }

  const room = rooms.find(r => r.id === roomId);

  if (!room) {
    return res.status(404).json({ message: "Room not found" });
  }

  if (room.isBooked) {
    return res.status(400).json({ message: "Room already booked" });
```

```
  }

  room.isBooked = true;
  room.customer = customerName;

  res.status(201).json({
    message: "Room booked successfully",
    room
  });
});

// GET /rooms/:id → View booking details
app.get('/rooms/:id', (req, res) => {
  const roomId = parseInt(req.params.id);
  const room = rooms.find(r => r.id === roomId);

  if (!room) {
    return res.status(404).json({ message: "Room not found" });
  }

  res.json(room);
});

// DELETE /rooms/:id → Cancel booking
app.delete('/rooms/:id', (req, res) => {
  const roomId = parseInt(req.params.id);
  const room = rooms.find(r => r.id === roomId);

  if (!room) {
    return res.status(404).json({ message: "Room not found" });
  }

  if (!room.isBooked) {
    return res.status(400).json({ message: "Room is not booked" });
  }

  room.isBooked = false;
  room.customer = null;

  res.json({
    message: "Booking cancelled successfully",
    room
  });
});
```

```
// Start server
const PORT = 3000;

app.listen(PORT, () => {
  console.log("Server is running on port " + PORT);
});
```

OUTPUT:



Task 4 – Online Voting API
Task:
Create a RESTful API for an online voting system.
Instructions:
• Endpoints required:
o GET /candidates → List candidates
o POST /vote → Cast a vote
o GET /results → View voting results
• Ensure one vote per user using in-memory validation.
Expected Output:
• API simulating a secure online voting process.

PROMPT:

//Design and implement a RESTful API for an online voting system that allows users to participate in a secure and controlled voting process. The API should include endpoints to retrieve a list of candidates (GET /candidates), cast a vote (POST /vote), and view the current voting results (GET /results). The system must enforce a strict rule of one vote per user by using in-memory validation to track users who have already voted and prevent duplicate submissions. Proper error handling should be implemented for cases such as invalid candidate IDs or repeated voting attempts. The final API should simulate a simple yet secure online voting environment, ensuring fairness and correctness in vote counting.

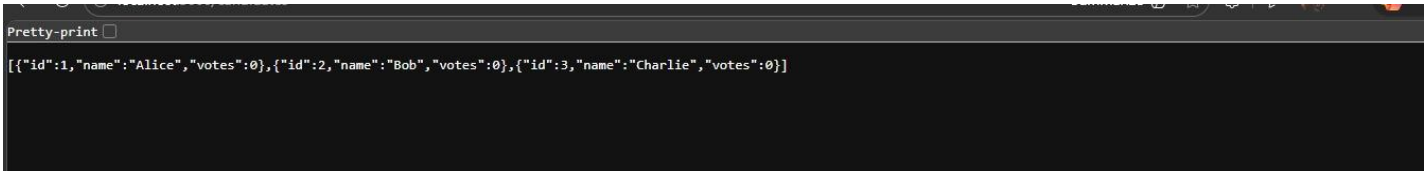
CODE:

```
const express = require('express');
const app = express();
```

```
app.use(express.json());

// In-memory data
let candidates = [
  { id: 1, name: "Alice", votes: 0 },
  { id: 2, name: "Bob", votes: 0 },
  { id: 3, name: "Charlie", votes: 0 }
];

// Store users who have voted
let votedUsers = new Set();

// GET /candidates → List candidates
app.get('/candidates', (req, res) => {
  res.json(candidates);
});

// POST /vote → Cast a vote
app.post('/vote', (req, res) => {
  const { userId, candidateId } = req.body;

  // Validate input
  if (!userId || !candidateId) {
    return res.status(400).json({ message: "userId and candidateId are required" });
  }

  // Check if user already voted
  if (votedUsers.has(userId)) {
    return res.status(403).json({ message: "User has already voted" });
  }

  // Find candidate
  const candidate = candidates.find(c => c.id === candidateId);
  if (!candidate) {
    return res.status(404).json({ message: "Candidate not found" });
  }

  // Cast vote
  candidate.votes += 1;
  votedUsers.add(userId);

  res.json({ message: "Vote cast successfully" });
});

// GET /results → View results
```

```
app.get('/results', (req, res) => {
  res.json(candidates);
});

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

OUTPUT:



Task 5 – Teacher Management API

Task:

Develop a RESTful API to manage teacher records.

Instructions:

• Endpoints required:

o GET /teachers → List all teachers

o POST /teachers → Add a new teacher

o PUT /teachers/{id} → Update teacher details

o DELETE /teachers/{id} → Remove a teacher

• Ensure proper error handling and JSON responses.

Expected Output:

• Functional API for managing teacher information.

PROMPT:

//Design and implement a RESTful API for an online voting system that allows users to participate in a secure and controlled voting process. The API should include endpoints to retrieve a list of candidates (GET /candidates), cast a vote (POST /vote), and view the current voting results (GET /results). The system must enforce a strict rule of one vote per user by using in-memory validation to track users who have already voted and prevent duplicate submissions. Proper error handling should be implemented for cases such as invalid candidate IDs or repeated voting attempts. The final API should simulate a simple yet secure online voting environment, ensuring fairness and correctness in vote counting.

CODE:

```
const express = require('express');
const app = express();
app.use(express.json());
```

```
// In-memory data
let candidates = [
  { id: 1, name: "Alice", votes: 0 },
  { id: 2, name: "Bob", votes: 0 },
  { id: 3, name: "Charlie", votes: 0 }
];

// Store users who have voted
let votedUsers = new Set();

// GET /candidates → List candidates
app.get('/candidates', (req, res) => {
  res.json(candidates);
});

// POST /vote → Cast a vote
app.post('/vote', (req, res) => {
  const { userId, candidateId } = req.body;

  // Validate input
  if (!userId || !candidateId) {
    return res.status(400).json({ message: "userId and candidateId are required" });
  }

  // Check if user already voted
  if (votedUsers.has(userId)) {
    return res.status(403).json({ message: "User has already voted" });
  }

  // Find candidate
  const candidate = candidates.find(c => c.id === candidateId);
  if (!candidate) {
    return res.status(404).json({ message: "Candidate not found" });
  }

  // Cast vote
  candidate.votes += 1;
  votedUsers.add(userId);

  res.json({ message: "Vote cast successfully" });
});

// GET /results → View results
app.get('/results', (req, res) => {
```

```
  res.json(candidates);
});

// Start server
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Server running on port ${PORT}`);
});
```

OUTPUT: